

LANGCHAIN + LANGGRAPH

Practical Notes — From LLM Chains to Stateful Agent Workflows

1. What is LangChain?

LangChain is a framework for building applications around language models. Instead of calling an LLM as a single isolated function, LangChain lets you connect prompts, models, parsers, tools and other components into reusable workflows.

Core idea

Input → Prompt → Model → Output Parser → Final Output

- Prompt: defines what the model should do.
- Model: generates the response.
- Output parser: converts model output into a useful format.
- Runnable components: allow different steps to be composed into a chain.
- Tools: let an application perform actions outside the model itself.

2. PromptTemplate

PromptTemplate creates reusable prompts with variables. This avoids writing the same prompt structure repeatedly.

```
from langchain_core.prompts import PromptTemplate

prompt = PromptTemplate(
    template="Write a joke about {topic}",
    input_variables=["topic"]
)

result = prompt.invoke({"topic": "AI"})
```

3. Chat Models

A chat model receives structured messages and returns an AI-generated response. For example, ChatOpenAI can be used as the model component in a LangChain workflow.

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(
    model="gpt-4o-mini"
)
```

Remember:

LangChain is mainly about **composing components into useful application workflows**.

LANGCHAIN: CHAINS & RUNNABLES

Building blocks for connecting multiple operations

4. Output Parsers

Models often return message objects. Output parsers help convert those responses into a simpler representation.

```
from langchain_core.output_parsers import StrOutputParser

parser = StrOutputParser()

chain = prompt | model | parser
result = chain.invoke({"topic": "AI"})
```

5. RunnableSequence

A sequence runs components one after another. The pipe operator `|` is a common way to compose them.

Prompt → Model → Parser

The output of one component becomes the input of the next component.

6. RunnableParallel

RunnableParallel allows multiple runnables to execute from the same input. It is useful when different pieces of information need to be generated independently.

```
from langchain_core.runnables import RunnableParallel

parallel = RunnableParallel({
    "joke": prompt | model | parser,
    "topic": lambda x: x["topic"]
})

result = parallel.invoke({"topic": "AI"})
```

7. RunnablePassthrough

RunnablePassthrough forwards the original input without changing it. It is useful when the original value must remain available alongside another result.

8. Chain Mental Model

User Input

↓

PromptTemplate

↓

Chat Model

↓

Output Parser

↓

Application Output

This linear style works well for straightforward tasks. When workflows need state, branching, loops or agent control, LangGraph becomes useful.

LANGGRAPH FUNDAMENTALS

Moving from chains to graph-based workflows

9. What is LangGraph?

LangGraph is designed for building stateful workflows and agent systems as graphs. A graph represents how execution moves between different pieces of work.

The three most important concepts are:

- **State:** information shared across the workflow.
- **Nodes:** functions that perform work and can update state.
- **Edges:** connections that determine which node runs next.

10. Basic Graph Flow

START → Node A → Node B → END

Unlike a simple chain, a graph can contain branches and loops. The next node can depend on the current state or the result of a previous node.

11. State

State is the shared data structure carried through graph execution. A TypedDict is one common way to describe the expected state fields.

```
from typing import TypedDict

class State(TypedDict):
    message: str
    count: int
```

12. Nodes

A node is normally a Python function. It receives the current state and returns updates to that state.

```
def process(state: State):
    return {
        "message": state["message"] + " processed",
        "count": state["count"] + 1
    }
```

13. Building the Graph

```
from langgraph.graph import StateGraph, START, END

graph = StateGraph(State)

graph.add_node("process", process)
graph.add_edge(START, "process")
graph.add_edge("process", END)

app = graph.compile()
```

LANGGRAPH: ROUTING & AGENTS

Conditional execution, loops and decision-making

14. Conditional Edges

A conditional edge allows the graph to choose the next node according to the current state.

START → Agent → Decision → Tool OR Final Response

```
def route(state):
    if state["count"] > 2:
        return "finish"
    return "process"

graph.add_conditional_edges(
    "agent",
    route,
    {
        "process": "process",
        "finish": END
    }
)
```

15. Loops

Graphs can return to an earlier node. This makes it possible to create iterative workflows such as generate → check → improve → check again.

Generate → Check →

■ if incorrect ■ if correct

Improve END

16. Tools inside an Agent Workflow

An agent can decide that it needs an external tool. The tool can perform an action such as retrieving information, calling an API, querying a database or running another operation.

User → Agent → Decide → Tool → Result → Agent → Final Answer

17. LangChain + LangGraph Together

LangChain can provide the components used inside a graph, while LangGraph controls the larger workflow. For example, a LangGraph node can call a LangChain prompt/model chain.

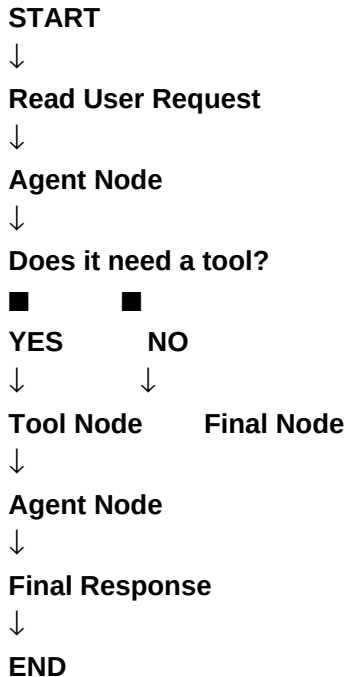
LangChain: components and model/application building blocks.

LangGraph: stateful orchestration and workflow control.

BUILDING A SIMPLE AGENT GRAPH

A practical structure to understand before larger projects

18. Example Architecture



19. Example State

```
class AgentState(TypedDict):
    user_input: str
    response: str
    tool_result: str
    needs_tool: bool
```

20. Combining Prompt + Model + Parser

```
prompt = PromptTemplate(
    template="Answer this request: {question}",
    input_variables=["question"]
)

chain = prompt | model | StrOutputParser()

def agent_node(state):
    answer = chain.invoke({
        "question": state["user_input"]
    })
    return {"response": answer}
```

21. Important Development Habit

Build the smallest graph first. Verify state updates and node transitions before adding tools, memory, multiple agents or complex routing.

22. Project Ideas

- Research assistant with search and summarization nodes.
- Document question-answering workflow.
- Coding assistant with tool execution and review nodes.
- Customer-support workflow with classification and escalation.
- Multi-agent workflow where different nodes handle different responsibilities.

LANGCHAIN + LANGGRAPH ROADMAP

What to learn after the fundamentals

23. Recommended Learning Order

Step 1 — LangChain Basics

PromptTemplate → Chat Models → Output Parsers → Runnable → Chains

Step 2 — LangChain Tools

Tool definitions → tool calling → external APIs → retrieval/database tools

Step 3 — LangGraph Basics

State → StateGraph → Nodes → Edges → START/END → Compile → Invoke

Step 4 — Graph Control

Conditional edges → branching → loops → state updates → error handling

Step 5 — Agent Workflows

Model decision → tool execution → tool result → model decision → final response

Step 6 — Advanced Systems

Memory/checkpointing → human-in-the-loop → multi-agent graphs → production workflows

24. Final Mental Model

LangChain gives you the building blocks.

LangGraph connects those building blocks into stateful workflows.

Agent = model + instructions + tools + state + decision flow.

25. One-Line Revision

If a task is mostly **Input** → **Prompt** → **Model** → **Output**, a LangChain chain may be enough. If the task needs **state**, **decisions**, **branching**, **loops** or **multiple steps**, think in terms of LangGraph.

Next target: build one small LangGraph agent yourself, then add a tool and conditional routing.